

## PROGETTO ESAME PROGRAMMAZIONE ORIENTATA AGLI OGGETTI – SUDOKU

Il gioco che ho scelto di realizzare è il Sudoku, un gioco di logica che mi piace molto e che per questo ho deciso di sviluppare.

Nel gioco tradizionale l'obiettivo del giocatore è quello di riempire tutte le 81 caselle presenti nella griglia con numeri da 1 a 9, senza ripetere più di una volta lo stesso numero in ogni riga, colonna e blocco 3x3.

Il gioco da me implementato vuole riprendere l'originale, quindi le regole non cambiano. Essendo però realizzato in formato digitale, è possibile inserire il proprio nome e registrare il tempo di esecuzione, questi dati verranno poi inseriti all'interno di una classifica. Inoltre, è possibile interrompere una partita, salvarla e riprenderla successivamente.

Sono previsti tre livelli di difficoltà: facile, medio e difficile. Questi livelli si distinguono per il numero di caselle libere presenti: il livello facile presenta 45 caselle libere, mentre sono 53 per quello medio e arrivano a 61 per quello difficile.

Nel momento in cui la griglia viene completata correttamente, compare un pop-up che indica al giocatore la vittoria; i dati del giocatore vengono salvati e inseriti nella classifica. A questo punto si può cominciare una nuova partita.

### Istruzioni per giocare

Quando il programma viene avviato compare prima un'intestazione e successivamente un campo di testo nel quale inserire il nome. Appena sotto due radio button che permettono di decidere se avviare una nuova partita o riprenderne una salvata; se si decide di avviarne una nuova, si attiva il menu a tendina che permette di scegliere il livello di difficoltà.

Una volta scelto il livello si può avviare il gioco con l'apposito tasto "Avvia gioco". A fondo pagina ci sono i tasti "Regole" e "Classifica", che permettono di visualizzare rispettivamente un pop-up con le regole e uno con la classifica.

The screenshot shows a window titled "Avvia Sudoku". At the top, the word "SUDOKU!" is displayed in large, bold, yellow letters. Below it, there is a text input field preceded by the label "Inserisci il tuo nome:". Underneath the name field, there are two radio buttons: "Nuova partita" (which is selected) and "Riprendi partita". Below these, there is a dropdown menu currently showing "Facile". At the bottom center, there is a button labeled "Avvia gioco!". At the very bottom, there are two buttons: "Regole" on the left and "Classifica" on the right.

Nel caso in cui non venga selezionata la modalità di gioco o non venga inserito il nome, vengono mostrati degli avvisi tramite pop-up.

Una volta avviata la partita compare la schermata di gioco, che presenta di nuovo un'intestazione, mentre nella parte centrale si sviluppa il vero e proprio Sudoku.

A sinistra abbiamo la griglia del Sudoku, mentre a destra si trovano il tastierino numerico, che permette di inserire e cancellare i numeri, selezionando prima la cella e poi sul numero, e il timer. Se il numero che si vuole inserire è già presente nella stessa riga, colonna o blocco 3x3, compare un messaggio di avviso.

A fondo pagina ci sono i tasti "Regole", "Classifica" e "Esci". I primi due permettono di visualizzare rispettivamente un pop-up con le regole e uno con la classifica. Il tasto "Esci" mostra invece una richiesta di conferma; se si conferma l'uscita, compare un ulteriore pop-up che chiede se si desidera salvare la partita.



## Architettura

Il progetto è organizzato secondo un'architettura a strati, separando la componente grafica dalla logica di gioco e dalle funzionalità di persistenza e supporto. Questa organizzazione viene concretizzata grazie alla suddivisione in diversi package:

- GUI, contiene i file che riguardano la parte grafica del progetto (Schermata Avvio, Schermata Gioco, Pop Up);
- Struttura Gioco, contiene i file che gestiscono la logica e la vera e propria creazione del gioco (Logica Gioco, Griglia Sudoku, Cella, Genera Sudoku, Soluzione Sudoku, Classifica);
- Utility, contiene i file che svolgono un ruolo di supporto al progetto (Gestione File Text, Dati, Costanti);
- Sudoku, contiene il file Main che permette l'avvio del programma.

## Modello di dominio

Gli elementi principali del mio programma sono:

- Cella → rappresenta una casella del Sudoku e contiene le sue coordinate, il numero presente e l'attributo fissa, che determina se essa possa o meno essere modificata.
- Griglia Sudoku → rappresenta la griglia composta da 81 celle, ricerca quelle vuote e controlla se un numero può essere inserito in una data cella.
- Genera Sudoku → si occupa di creare il Sudoku, riempie la griglia vuota e toglie un numero di caselle in base alla difficoltà scelta.
- Soluzione Sudoku → verifica che il Sudoku generato abbia una soluzione e che questa sia unica.
- Logica Gioco → crea una nuova partita e gestisce l'inserimento o la rimozione dei numeri nelle celle da parte dell'utente; inoltre, controlla quando una partita termina.
- Classifica → raccoglie e ordina i dati in base al livello e al tempo.
- Dati → gestisce i dati del giocatore, cioè nome, livello e tempo impiegato.
- Gestione File Text -> salva su un file di testo la classifica e la ricarica quando richiesto; inoltre, salva la partita in corso e la ricarica quando la si vuole riprendere.
- Schermata Avvio -> gestisce la schermata iniziale.
- Schermata Gioco -> gestisce la schermata di gioco, dove si svolge la partita.
- Pop Up -> gestisce la visualizzazione dei messaggi e delle finestre di supporto.

## Diagramma delle classi UML

Il diagramma delle classi UML-like del progetto è riportato in un file separato.

## Scelte implementative

Ho scelto di utilizzare una matrice 'Cella[][]' per rappresentare la griglia, perché mi permette di rappresentare ogni singola cella mantenendo i relativi attributi, come le coordinate, il numero presente e l'attributo fissa, attributo aggiunto alla classe 'Cella' per distinguere le celle iniziali del Sudoku da quelle modificabili e impedire all'utente di modificare la griglia di partenza.

Ho separato le classi "Genera Sudoku" e "Soluzione Sudoku" perché svolgono due compiti differenti. "Genera Sudoku" contiene i metodi necessari per creare il Sudoku, mentre "Soluzione Sudoku" si occupa di verificare la soluzione e la sua unicità. In questo modo ogni classe ha una funzione specifica.

Avevo bisogno di uno strumento di misurazione del tempo per poter creare una classifica, quindi ho utilizzato il Timer di Swing, perché permette di aggiornare il tempo a intervalli regolari, nel mio caso ogni secondo, e di mostrare al giocatore il tempo trascorso durante la partita.

## **Librerie**

Non ho utilizzato librerie esterne, ma solo librerie Java, in particolare:

- java.util → List, ArrayList, Random, Collections;
- javax.swing → componenti per la realizzazione dell'interfaccia grafica;
- java.awt → layout, dimensioni, font, colori e altri elementi grafici;
- java.io → lettura e scrittura dei file .txt.

## **Meccanismi e algoritmi**

Gli algoritmi più importanti del progetto sono riportati qui sotto, mentre nel codice ogni metodo ha un commento che specifica quale sia la sua funzione.

Generazione del Sudoku (riempiMatrice() - sudokuPerLivello())

La griglia viene riempita un numero alla volta, esso viene scelto in modo casuale da una lista di supporto contenente numeri da 1 a 9 mescolati in modo casuale. Se il numero non è già presente nella stessa riga, colonna o blocco 3x3 viene considerato valido e si continua con il riempimento della griglia, se si arriva a un punto in cui non è possibile proseguire, il metodo torna indietro, svuota la cella precedente e prova un altro numero, continuando fino a completare la griglia. Per creare la struttura del Sudoku vengono scelte casualmente alcune celle che vengono svuotate, il numero di celle varia in base al livello selezionato.

Controllo dell'unicità della soluzione (analizza(...))

All'interno della classe Soluzione Sudoku il metodo "analizza" racchiude due compiti: ricerca della soluzione per il Sudoku e controllo soluzione unica. In primo luogo il metodo controlla che il Sudoku sia completo, in tal caso abbiamo una soluzione e la griglia può essere considerata risolvibile. Se la griglia non dovesse essere completa, il metodo prova a riempire le celle vuote con lo stesso meccanismo usato in Genera Sudoku, facendo attenzione a non modificare le celle fisse. Se si trova una soluzione viene soddisfatta la ricerca della risolvibilità del gioco, mentre per verificare che la soluzione sia unica, il metodo continua la ricerca anche dopo aver trovato una prima soluzione. Se si trova una seconda soluzione, la condizione di unicità non viene soddisfatta e

quindi è necessario creare un nuovo Sudoku, altrimenti può essere proposto al giocatore.

#### Controllo degli inserimenti (inserimentoPossibile(...))

Ogni volta che il giocatore cerca di inserire un numero in una cella, vengono considerate le coordinate di quella cella e viene confrontata con tutte le celle che si trovano sulla sua stessa riga, sulla sua stessa colonna e nello stesso blocco 3x3 per evitare che il numero sia già presente; se il controllo va a buon fine allora il numero può essere inserito.

#### Controllo della fine della partita (terminePartita())

Per verificare che una partita sia terminata si controlla che non ci siano più celle vuote e che i numeri inseriti dal giocatore corrispondano a quelli presenti nella soluzione creata all'inizio.